ExportPilot — India-UAE Trade Copilot

Tagline: Speak your deal. ExportPilot quotes it, checks the risk, and drafts the buyer message.

Category: Best Business AI Agent (Dubai Create AI Agents Championship)
Demo: Streamlit | Voice: Tamil, Hindi, English | Cost: ₹0

---

1. Day-1 setup (you do this)

1. Install Python 3.11+, VS Code or Antigravity, Git.
2. Make a free Gemini API key at Google AI Studio.
3. Create a GitHub repo `exportpilot` (public or private).
4. Open the empty folder in Antigravity.
5. Create `.env` with `GEMINI_API_KEY=...` and `GEMINI_MODEL=<current Flash model name from AI Studio>`.
6. Add `.env` to `.gitignore`.
7. Register on the championship page (dubaichamberdigital.com/en/create-ai-agents).

2. Rules file for Antigravity

Save as the workspace rules file (Antigravity rules / AGENTS.md):

```
PROJECT: ExportPilot, an AI agent that
helps small Indian exporters quote and
manage deals with buyers (focus: India to
UAE).

HARD RULES
- Zero cost: only free tiers and open-
source libraries. No paid APIs.
- Stack: Python 3.11, Streamlit (UI),
LangGraph (agent flow), SQLite (storage),
  google-genai (Gemini), requests,
feedparser.
- Original code only. Do not copy existing
repositories.
- Secrets only in .env. Never hardcode
keys. Never commit .env.
- The agent must ACT (create quote, save
record, draft message) but must NEVER
  send or finalize anything without an
explicit human approval click.
- Use synthetic/demo data only. No real
customer data.
- Every tool function gets a unit test. Run
tests before saying a task is done.
- Keep files small and clear. Add short
docstrings.
- All UI text, README and docs in English
(voice input also accepts Tamil/Hindi).
```

```
- If a free-tier limit is hit, fail
gracefully with a clear message.

WORKFLOW
- Plan first, list steps, then build. Work
one milestone at a time.
- After each milestone: run the app/tests,
report what works and what does not.
```

3. Folder structure

```
exportpilot/
  app.py              # Streamlit UI
  agent/
    graph.py          # LangGraph flow
    tools.py          # fx, news, buyer
norms, quote, save
    voice.py          # audio -> text
(Gemini)
    prompts.py
  data/
    buyer_norms.json  # hand-curated
customs per country (UAE first)
    exportpilot.db    # SQLite (auto-
created)
  tests/
  requirements.txt
  README.md
  .env  (not committed)
```

4. Agent flow

`understand request -> FX rate -> trade news/risk -> buyer-country norms -> draft quote + buyer message -> HUMAN APPROVAL -> save`

Tools:

- `get_fx(base, quote)`: https://open.er-api.com/v6/latest/INR (free, no key)
- `get_trade_news(country)`: RSS feeds (DGFT, PIB, Gulf business news), summarized by Gemini
- `get_buyer_norms(country)`: local JSON (working week, greeting style, payment terms, holidays)
- `draft_quote(...)`: price, currency, validity, FX-risk note
- `save_deal(...)`: SQLite

5. Voice input (Tamil, Hindi, English)

- UI: `st.audio_input("Speak your request")` returns audio bytes.
- `voice.py`: send audio to Gemini with this instruction and parse JSON:

```
Transcribe this audio exactly. Detect the
language (ta, hi, or en).
Then translate to English. Return ONLY
```

```
JSON:
{"language": "...", "transcript": "...",
"english": "..."}
```

- The agent works on the English text; show the original transcript to the user.
- Always add a typed-text fallback box (voice can fail on noisy audio).
- Test with 10 Tamil, 10 Hindi and 10 English sample sentences and record accuracy for the pitch.
- Privacy note: free-tier API inputs may be used by the provider. Use demo audio only, and say so in the pitch. Check current terms.

6. Master prompt #1 (paste into Antigravity, Planning mode)

```
Read the rules file. Build Milestone 1 of
ExportPilot.

Scaffold the folder structure. Create
requirements.txt (streamlit, langgraph,
google-genai, requests, feedparser, python-
dotenv, pytest).
Implement agent/tools.py with get_fx and
get_buyer_norms plus data/buyer_norms.json
(UAE, USA, Germany, UK, Saudi Arabia: 5
fields each). Add pytest tests.
Build a minimal Streamlit app.py with a
```

```
text box: user types
"Quote 500 cotton shirts to a Dubai buyer"
and the app shows the live
INR/AED rate and the buyer-country norms.
Run the tests and the app. Report results.
Do not build anything else yet.
```

7. Next prompts (one per milestone)

M2 Agent: "Build the LangGraph flow in agent/graph.py with the steps in section 4, using Gemini for understanding and drafting. Output a structured quote and an English buyer email. Add tests with mocked tools."

M3 Risk: "Add get_trade_news with RSS feeds, summarize tariff/policy items relevant to the destination country, and show a risk level (low/medium/high) with reasons."

M4 Voice: "Add agent/voice.py per section 5 and the st.audio_input widget with a typed fallback. Show detected language, transcript and English text."

M5 Approval + save: "Add a human approval step: show the draft, require an Approve button, then save to SQLite and show a deals history table. Nothing is finalized before approval."

M6 Polish: "Add a clean UI, an impact panel (time to quote, FX loss avoided estimate), error handling, a

README with architecture diagram, and deploy instructions for Hugging Face Spaces."

8. Your real-world tasks (Antigravity cannot do these)

- Talk to 2-3 real exporters (garments, leather, food, auto parts). Ask: how do you quote, what goes wrong, what does a mistake cost.
- Record their feedback on video (with permission).
- Time a manual quote vs ExportPilot for the pitch numbers.
- Write the pitch deck (problem, user, demo, numbers, scale, data protection).
- Record a 60-90 second demo video: one exporter, one voice request, quote ready.

9. Checklist before submission

- [ ] Original code, no copied repos
- [ ] Human approval step visible in demo
- [ ] Data protection explained
- [ ] Real-user quote or pilot result
- [ ] Measured time saved
- [ ] Prototype link, pitch deck, demo video, all in English

- ( ) Submitted at least a week before the deadline